HashiCorp Certified: Terraform Associate（004）　目標 4a・4b　#06

resourceとdata・参照

resource ブロックの型・名前・引数・メタ引数、data ブロックとデータソースを読むタイミング、
terraform_remote_state、名前つきの値、属性の参照・スプラット式・リソースアドレス・暗黙の依存関係

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の Create and manage resources overview・Configure a resource・Query infrastructure data・Resource Address Reference・References to Named Values・Splat Expressions・Remote State）を書いた。英語の原文は『』で引用した

第1章

resource と data

作って管理するか、読むだけか。

resource と data のちがい

```
data "aws_ami" "web" {          # 読むだけ
  most_recent = true
  owners      = ["self"]
  filter {
    name   = "tag:Component"
    values = ["web"]
  }
}

resource "aws_instance" "web" {  # 作って管理
  ami           = data.aws_ami.web.id
  instance_type = "t2.micro"
}
```

	resource	data
すること	作る・変える・消す	読むだけ
参照	aws_instance.web.id	data.aws_ami.web.id
destroy	実物が消える	何も消えない

> 『Data sources fetch data from the provider, but do not create or modify resources.』

出典：Configure a resource／Query infrastructure data

resource ブロックの中身

```
resource "aws_db_instance" "example" {
  # ↑型（何を・どの引数）   ↑ローカル名 → アドレス aws_db_instance.example
  count    = 2               # メタ引数（Terraform の管理のしかた）
  provider = aws.west

  engine         = "postgres" # 引数（プロバイダーが決める）
  instance_class = "db.t3.micro"

  timeouts {                 # 対応する型だけの子ブロック
    create = "60m"
    delete = "2h"
  }
  lifecycle {
    prevent_destroy = true
  }
}
```

型名の先頭 → プロバイダー
aws_instance の aws。別の設定は provider = aws.west

メタ引数
count・for_each・provider・depends_on・lifecycle

apply がすること
ないものを作る・消えたものを消す・その場で更新・できなければ作り直し・ステート更新

> ローカルだけのリソース：random_id・tls_private_key（ステートにだけ値がある）

出典：Configure a resource／Create and manage resources overview

データソースはいつ読む？

引数がすべて計画で分かる
plan の前の refresh で読む → 計画で値が使える

引数が、この計画で変わるリソース・適用で決まる値に依存
apply まで延ばす → 計画では computed（known after apply）

延ばすと、その値を使うリソースも apply まで決まらない。

データソース	何をするか
aws_ami・aws_vpc など	外で作られたものを条件で探す
aws_iam_policy_document	IAM ポリシーの JSON を作る（手元で計算）
local_file	手元のファイルを読む
terraform_remote_state	ほかの設定のステートの出力値を読む

data にも count・for_each・provider・depends_on・precondition/postcondition が使える。

出典：Query infrastructure data（Data source behavior／Specialized data sources）／Remote State

第2章

参照とアドレス

名前で読む。参照が順番を決める。

名前つきの値

種類	書き方	メモ
管理するリソース	aws_instance.web.id	ほかの形に合わない名前はリソースとみなす
データソース	data.aws_ami.web.id	data. を付ける
入力変数	var.region	型の制約に変換された値
ローカル値	local.name	定義は locals ブロック
子モジュールの出力	module.network.subnet_ids	count ならリスト、for_each ならマップ
ファイル・ワークスペース	path.module・path.root・path.cwd・terraform.workspace	path.module 以外はルートだけで
ブロックの中だけ	count.index・each.key・each.value・self	入力変数とは別のもの

本物のオブジェクトではない：『you cannot use aws_instance in a for expression to iterate over every AWS instance resource』

出典：References to Named Values

属性の参照とスプラット

リソース	値の形	読み方
count・for_each なし	オブジェクト	aws_instance.web.id
count	リスト	[0].id・[*].id
for_each	マップ	["a"].id・for 式・values(…)[*].id

引数（ami）も属性（id）も同じ書き方で読める。

```
var.list[*].id
# = [for o in var.list : o.id]

aws_instance.example.ebs_block_device[*].device_name

[for v in aws_instance.example : v.id]   # for_each
values(aws_instance.example)[*].id       # for_each
```

スプラットはリスト・セット・タプルだけ。マップには for 式。

出典：References to Named Values（References to Resource Attributes）／Splat Expressions

暗黙の依存関係

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
resource "aws_subnet" "a" {
  vpc_id     = aws_vpc.main.id   # ← 参照
  cidr_block = "10.0.1.0/24"
}
```

↓ Terraform が式を読んで

aws_vpc.main → aws_subnet.a
VPC が先。依存しないものは並列

『an expression in a resource argument that refers to another managed resource creates an implicit dependency』

local 値を通した間接の参照でも見つける。

参照では分からない依存（IAM ポリシーの付与を待つなど）は depends_on で明示（#09）。

出典：References to Named Values（Named Values and Dependencies）／Configure a resource（Specify resource dependencies）

リソースアドレス

```
[module path][resource spec]

module.foo[0].module.bar["a"].aws_instance.web[2]
└ foo の 1 つ目 ┘└ bar のキー a ┘└ web の 3 つ目 ┘
```

アドレス	指すもの
aws_instance.web[3]	count の 4 つ目（0 から数える）
aws_instance.web["example"]	for_each のキー example
aws_instance.web	全インスタンス
module.foo	モジュールの中の全部

モジュールのパスを省くとルートモジュールだけ（0.12 以降）。

使う場所：state list・state show・-target・-replace・import・moved など。

sensitive な属性を output に使うなら、output にも sensitive = true。ステートには平文で残る。

出典：Resource Address Reference／References to Named Values（Sensitive Resource Attributes）

本番での問われ方

ねらい	見るところ
resource と data	作る vs 読むだけ、data. の付け方、外で作ったものは data で読む
resource の書き方	型と名前、アドレス、引数とメタ引数、timeouts
データソースのタイミング	plan で読む・apply まで延ばす条件
参照	var・local・module・data・path、count はリスト、for_each はマップ
スプラット	[*] ＝ for 式、マップには使えない
アドレスと依存	[N] と ["KEY"]、module の入れ子、参照で暗黙の依存

出典：Exam Content List（004）4a・4b／本資料の問題演習（全 30 問）